



Hack<IT>



Root Cause Analysis

Root Cause Analysis

Storyline

You are an ecologist working on the planet Photospheria, and your job is to build a biological sample from the planet's plant life and bring it back to Earth for study.

The plant life on Photospheria works through interaction. Different plants affect each other depending on where they are placed. Some combinations cause new species to appear. Other combinations make it more difficult for plants to flourish. Because of this, what you choose to place together directly shapes which kinds of plants exist in your final sample.

Your main goal is to create the most useful possible sample for Earth's scientists.

Each unique species provides new information into how life functions on this planet. A more diverse collection offers a broader and more complete understanding of Photospherian biology. Therefore, your score is determined primarily by the number of different species you successfully cultivate and include in your final sample.

There is also a secondary goal: longevity.

The longer a species remains alive, the more reliable the data available for analysis back on Earth.

Your task is simple:

- Build the best possible sample of Photospherian plant life.
- Maximise the diversity of species in your collection and keep them alive for as long possible.

Fortunately, some of your ecological colleagues have gone through the trouble of creating various encyclopedias and notes of all the plants, animals and events you will encounter in your experiments. These notes, along with a deep dive of the mechanics you will work with can be found in the following pages.

Problem Overview

Basic Game Structure and Rules

- Your garden is an $N \times M$ grid on which you can place plants.
- Each *cell* (x-y coordinate location on the grid) can only contain one plant at a time, unless a plant's rules explicitly allow coexistence (e.g., the `subsurface_growth` special rule).
- Time is discrete and represented with ticks.
- On each tick you may choose to plant a minimum of zero and a maximum of twenty plants at specific coordinates within the grid.
- Your submission will be a sequence of instructions specifying which plants to place at which grid points. See the [Submission Format](#) section for more details.

- Most plants are not available for planting straight away. These plants are locked behind unlock conditions, which must be met before you can place them. Any attempt to place a plant before its unlock conditions have been met will be ignored.
- Different plants have different properties and spreading behaviours.
- Your objective is to strategically plant plants to unlock as many plant types as possible.
- Only the final state of your garden is used to calculate your score. See the [Scoring Function](#) section for more details on how your sample will be scored.

Problem Mechanics

Grid and Time

The garden is represented as an $N \times M$ grid. Each cell can contain a plant, subject to the game rules. The simulation runs for T ticks (days). On each tick, you may choose to plant one or more plants at specific coordinates within the grid however, you can only place one plant per cell. Participants must submit a solution file describing their planting actions throughout the simulation.

Grid Layout

The garden is represented by a coordinate system. **Example:**

- Grid Width (N): 100 cells
- Grid Height (M): 100 cells
- Valid X coordinates: 0 - 99
- Valid Y coordinates: 0 - 99

```
(0,0) -----> X
|
|
|
v
Y
```

The top-left corner is located at coordinate (0,0).

Ticks (Days)

The simulation progresses in discrete time steps called ticks. Tick 1 represents Day 1. Tick T represents the final day. Participants may schedule planting actions on any tick from 0 (initial setup) to $T-1$, final actions occur on the penultimate day, and scoring is done on the state of the world on the final day.

A planting action specifies:

- The tick at which to place.
- The plant index (matching the `index` value in the supported plant catalogue).
- The X coordinate.
- The Y coordinate.

Submission Format

Solutions must be submitted as a JSON file. The JSON contains an array of tick entries. Each tick entry contains:

- The tick number.
- A list of planting actions that should occur during that tick.

Example:

```
{
  "actions": [
    {
      "tick": 1,
      "plants": [
        {
          "plant_index": 6,
          "row": 0,
          "col": 0
        },
        {
          "plant_index": 2,
          "row": 20,
          "col": 20
        }
      ]
    },
    {
      "tick": 5,
      "plants": [
        {
          "plant_index": 6,
          "row": 5,
          "col": 5
        },
        {
          "plant_index": 2,
          "row": 15,
          "col": 15
        }
      ]
    }
  ]
}
```

Schema Definition

Each planting action follows the structure below:

```
{
  "index": 1,
  "row": 0,
  "col": 0
}
```

Constraints

- Coordinates must remain within the bounds of the grid.
- Ticks must be positive integers between 0 and T-1.
- Multiple plants may be scheduled during the same tick.
- Plant indices must match the `index` values in the supported plant catalogue provided for the challenge. See section [Plant Encyclopedia](#).
- Plants with unlock conditions will only place successfully if all of their unlock conditions are met.
- Plants can only be placed in soil; any other terrain type is uninhabitable.
- Only one plant may be placed on a cell. If your action dictates that a plant must be placed on an occupied cell, then the existing plant on that cell will be replaced by your new plant.
- You may only plant 20 plants per tick. Only the first 20 plants in your list for each tick will be planted.

Plant Encyclopedia

Each plant species is defined by a set of basic properties that determine how it grows, spreads, and competes within the ecosystem. Plants are recorded with the following basic properties:

- **Time to maturity**
The amount of time a plant requires before reaching maturity. A plant can only begin competing for nutrient capacity in a cell once it has matured. You can find out more about nutrient capacity in the [Soil](#) section below. Plants that cast shade or have area effects only start once they are mature.
- **Rate of spread**
How long it takes for the plant to spread to surrounding cells, measured in ticks.
- **Spread mechanism**
The method by which the plant propagates. Different spread mechanisms interact differently with environmental features and animal species, which may help or hinder the spread of the plant.
- **Spread type**
The shape or direction in which the plant propagates. Examples: Moore, Von Neumann, Row, Column, Cross-hatch.
- **Spread range**
How far the plant propagates in the specified pattern at each step of a spread action.

- **Root type**
Shallow-root, deep-root, and none. These have an effect on unlock conditions.
- **Invasiveness hierarchy**
This rank determines which plants will outcompete the current plant. The higher the rank, the more plants it can outcompete. If two plants want to occupy the same cell, the one with a higher rank wins.
- **Weaknesses**
Specific conditions under which a plant cannot spread, thrive or survive.
- **Special rules**
Unique abilities, or behaviours associated with a plant species.
- **Preferred soil**
A plant will only spread into cells of its preferred soil.

The full catalogue of plant species and their properties is provided in `plant_dataset.json`. The JSON schema for ingesting this data is defined in the [Plant Ingestion](#) section.

Each plant also belongs to a classification. These classifications form part of certain unlock conditions and can be found in `classifications.json`.

You will start off each level with access to only the following plants:

- Grass
- Rose Bush
- Lavender
- Dwarf Sunflower
- Oak Tree

It is up to you to strategically plant these to create the conditions in Photospheria needed to plant more plants and unlock as many species as possible. Refer to the [Plant Unlock Conditions](#) section for each plant's specific unlock conditions.

Photospheria Cell Types

Soil and cell types:

These constitute the cells in which you will be allowed to place plants. Each plant also has preferred soil types it can spread into.

Nutrient capacity is a resource that is consumed when a plant occupies a cell. Cells start with 100 nutrient points. While a plant occupies a cell, its nutrient points decrease at a rate of 1 point per tick. When the cell reaches 0 points, it is no longer inhabitable and the plant dies, leaving dead plant matter in the cell. A cell containing dead matter regains nutrient points at a rate of 1 point per tick, capped at 100.

Soil is divided into the following types, identified by numeric ID:

- 0 : Dirt
- 1 : Mud
- 2 : Clay
- 3 : Burnt

Photospheria also contains the following natural elements affecting its soil:

- **Water Feature**
Always surrounded by clay soil types.
- **Stone**
These cells are uninhabitable. Stone reeds can only sprout from cells adjacent to stone.
- **Path**
A terrain type. Plants cannot spread onto paths.
- **Cracks**
These cells form after earthquakes. New cracks may appear over time, altering the layout of the world. Skyvine can spread over cracks. Experiment with different submissions to gauge the formation of cracks due to earthquakes in the world.
- **Burnt Soil**
These cells surround Emberroot Trees. These harsh conditions make it difficult for almost all plants to spread into burnt soil.

Climate, Seasons, Plant Coverage and World Events

The world of Photospheria is subject to various environmental conditions that affect plant growth, spread, and survival.

- **World events**
The known world events are Rain, Drought, Ash Eclipse, and Earthquake. Natural events are the same as world events. These events are scheduled to occur at a specific tick, which is defined in the level file. They are referenced by plant unlock conditions. Example: Crystal Cactus requires a Drought event, Mire Bloom requires a Rain event, and Phoenix Bloom requires an Ash Eclipse.
- **Shade**
Certain plants cast shade, like Oak Tree, for example. Once such a plant reaches maturity, the cells surrounding it within a given radius are given the shaded state. The `shade_radius` special rule determines how far around the plant the shade condition extends. Shade prevents plants with a shade weakness (`no_shade_survival`, `no_shade_spread`) from spreading into a shaded cell. The `adjacent_shade_penalty` special rule applies when the plant with this rule exists on a cell that is shaded.

- **Seasons**

The seasons are Spring, Summer, Autumn, and Winter. Season changes are dictated in the level file, which specifies the new season name and the tick on which the change occurs. Seasons are deterministic and only exist to activate plant-specific rules, such as `no_winter_spread` or the `season_summer` conditional modifier.

- **Dead matter**

Dead matter occurs in a cell where a plant has died. That is, where the cell's nutrient points have fallen to 0. It is represented by a state flag on the cell. When a cell contains dead matter, it regains nutrient points at a rate of 1 point per tick (capped at 100). If a plant moves into a cell containing dead matter, the cell instead loses nutrient points at a rate of 0.5 per tick. Plants only feel the effects when nutrients are on 0 (resulting in plant death).

- **Coverage percentages**

Percentages used in unlock conditions and animal requirements refer to the total available cell coverage in the world. Example: "8% Coverage with Lavender" means that if the world had 100 cells, at least 8 of them must contain Lavender for the requirement to be met.

Animals

Among the plant life on Photospheria, there is an abundance of wildlife that also has a significant effect on how the plants grow, spread and thrive. Animals are either absent from the world or present. When certain conditions are met, an animal type becomes present in the world and starts affecting plants in certain ways. Once the conditions for a particular animal are no longer met, that animal will also no longer be present in your world.

The full list of animal species, their appearance requirements, and their effects is provided in `animals.json`. The JSON schema for ingesting this data is defined in the [Animal Interaction Data Structure](#) section.

Scoring Function

Your goal is to create a large and diverse sample, to give scientists back on earth the optimal amount of each plant type to study. Avoiding monocultures is therefore important, as greater plant diversity contributes to a healthier ecosystem.

Additionally, the older a plant becomes, the more scientists can learn about its growth and life cycle.

Diversity Score (Entropy)

Definitions:

- N = Total number of species types in the game
- n_i = Number of species i present on the grid
- $C = \sum n_i$ = Total number of plants present on the grid
- $p_i = \frac{n_i}{C}$ = Proportion of species i

Then the standard diversity (spread) score is entropy:

$$H = - \sum_{i=1}^N p_i \log_N(p_i)$$

where

$$0 \cdot \log_N(0) \equiv 0$$

H attains its maximum if all p_i are equal (the spread of species is uniform).

Sample Size Factor

The size of your sample is scored against the maximum obtainable sample size.

Definitions:

- C = Number of populated cells (same as before)
- $C_{max} = N \times M$ = Total number of cells on the grid
- α = Unknown scaling parameter

The sample size factor is then given by:

$$\left(\frac{C}{C_{max}} \right)^\alpha$$

And your **Main Score** is

$$H \left(\frac{C}{C_{max}} \right)^\alpha = \left(- \sum_{i=1}^N p_i \log_N(p_i) \right) \left(\frac{C}{C_{max}} \right)^\alpha$$

Secondary Longevity Score

Definitions:

- T = Total number of time ticks in the game
- l_{ij} = Lifespan (measured in ticks) of the plant on the cell with coordinates (i, j) . Set to 0 if the cell is empty. Here, i runs over rows and j over columns
- $C_{max} = N \times M$ = Total number of cells on the grid (same as before)
- k = Unknown scaling parameter

Note: α and k are used as scaling parameters, and these values are not shared.

The **Longevity Score** is a normalised sum over the lifespans of all plants in the final sample:

$$\left(\frac{1}{C_{max}}\right) \sum_i \sum_j \left(\frac{l_{ij}}{T}\right)^k$$

Final Score

The final score is a weighted sum of the main and longevity scores

$$\text{Final Score} = 0.8 \times (\text{Main Score}) + 0.2 \times (\text{Longevity Score})$$

Note:

- The final score is scaled by a large unknown constant to produce the score on the leaderboard.
- This function will only be applied to the state of your garden on the final tick. All plants that died before the final tick will not play a role in the scoring function.

Level Descriptions

Your experiments will be done in different stages to help you become familiar with certain concepts before scaling up into the final level.

Level 1: Greenhouse Study

You will grow your plants in a greenhouse to guard against animals and weather conditions however, seasons still apply. You will only work with a small subset of plants, and your aim is to grow a diverse garden. Because of the limited space, you will only be able to use a small subset of the plants.

Level 2: Garden Growth Study

Now that you have the hang of things, we will move outside into a small garden. Your plants are now exposed to the elements, and a few critters might come and say hi. To add to this, you also have a wider variety of plants available since you have more space.

Level 3: Park Potential

Let's kick it up a notch and take it out to a park where we introduce plant unfriendly terrain. Photospherian weather events might also be on the horizon as we venture further into nature.

Level 4: Forest

You are exposed to all the elements in Photospheria. There is a wide-open forest at your disposal. Use it wisely!

Game Ingestion

Plant Ingestion

You can find the data in `plant_dataset.json`. The dataset is a single JSON array of plant objects:

```
[
  { plant_object_1 },
  { plant_object_2 },
  ...
]
```

Each plant object has five top-level keys:

- `plant`
- `index`
- `growth`
- `preferred_soil`
- `rules`

Top-Level Structure

1. `plant` (string)

```
"plant": "Grass"
```

A unique identifier for the species.

- Used for referencing, logging, and debugging.

2. `index` (int)

```
"index": 1
```

A numeric identifier for the species.

3. `growth` (object)

Defines all intrinsic biological and simulation-relevant growth parameters.

```
"growth": {
  "time_to_maturity": number,
  "spread_rate": number,
  "spread_mechanism": string,
  "spread_type": string,
  "spread_range": number,
  "root_type": string,
  "invasiveness_rank": number,
  "conditional_modifiers": [optional array]
}
```

Field Definitions

`time_to_maturity` (int)

- Number of ticks required before the plant becomes "active".
 - Before this threshold, the plant does not spread or trigger effects.
-

`spread_rate` (int)

- The number of ticks that must pass before the plant can perform a spread action.
 - A lower value means the plant spreads more often.
-

`spread_mechanism` (string)

Defines the biological propagation mechanism.

Typical values:

- | | |
|------------|-----------------|
| • "Shoots" | • "Spores" |
| • "Seeds" | • "Pollination" |
-

`spread_type` (string)

Defines spatial propagation geometry. Common values:

- "VonNeumann" → 4-directional (N, S, E, W).
 - "Moore" → 8-directional (including diagonals).
 - "Row" → linear propagation along a row axis.
 - "Column" → linear propagation along a column axis.
 - "CrossHatch" → structured multi-axis expansion pattern.
-

`spread_range` (int)

- Determines the radius around the plant that is included in its `spread_type` pattern.
-

`root_type` (string)

Defines environmental dependency layer and plant interaction. Possible values:

- `"Shallow"` → surface-level dependency, sensitive to immediate conditions.
- `"Deep"` → stable, less sensitive, possibly stronger persistence.
- `"None"` → no root dependency (fungal-like or free-floating systems).

Used by environmental simulation layers (soil, terrain, nutrients, stability rules).

`invasiveness_rank` (int)

- Relative aggressiveness / ecological dominance score.
- Higher values imply:
 - Stronger spread pressure.
 - Higher competition strength.
 - Greater ecosystem impact.

Competition resolution

When two plants spread into the same cell, the plant that spreads into the cell last wins — since neither plant has reached maturity at that point, neither can compete using its invasiveness rank. The plant that occupied the cell first is therefore overwritten by the next plant to spread into it. Note that some special cases, such as Ashroot Bramble, which only spreads on burnt soil, are exceptions.

`conditional_modifiers` (optional array)

Applies dynamic modifications to growth parameters based on environment state.

```
"conditional_modifiers": [  
  {  
    "condition": string,  
    "spread_rate": number (optional override),  
    ... (future extensible overrides)  
  }  
]
```

Example:

```
{  
  "condition": "season_summer",  
  "spread_rate": 4  
}
```

Semantics

- If the given condition evaluates to TRUE:
 - Override specified fields in `growth`.
- If multiple modifiers apply:
 - The simulation applies them in a specific order of priority. You can play around and try to figure out which conditions have higher priority.

4. `preferred_soil` (array of int)

```
"preferred_soil": [0, 1]
```

Defines which soil types on which the plant can grow on.

- Each element is a numeric soil type identifier.
- Plants can only be placed on or spread into cells whose soil type is in this list.

The soil type identifiers are:

- | | |
|------------|-------------|
| • 0 : Dirt | • 2 : Clay |
| • 1 : Mud | • 3 : Burnt |

5. `rules` (object)

Defines ecological constraints and emergent behavior triggers.

```
"rules": {  
  "weaknesses": [ rule_object ],  
  "special": [ rule_object ]  
}
```

Rules are purely declarative. The simulation applies them at runtime.

`weaknesses` (array of rule objects)

Represents failure conditions or restrictions.

```
{  
  "type": string, ...optional parameters  
}
```

Common Weakness Types

- `"no_shade_survival"`
 - Plant cannot survive under shade conditions.
- `"no_shade_spread"`
 - Plant may exist but cannot spread in shade.

- "no_winter_spread"
 - Spread disabled during winter conditions.
- "die_if_neighbors_greater_than"
 - Defines a numeric parameter:


```
{
  "type": "die_if_neighbors_greater_than",
  "value": int
}
```
 - Plant dies if adjacent neighbour count exceeds the given threshold.
- "die_if_isolated"
 - Plant dies if it has no neighbours.
- "must_be_adjacent_to"
 - Requires environmental adjacency constraint:


```
{
  "type": "must_be_adjacent_to",
  "feature": string
}
```
 - Example features: "water", "rock_or_path"
- "shade_required"
 - Requires shaded environment to survive/grow.
- "must_be_burnt_soil"
 - Requires burnt soil to survive.
- "no_adjacent_plants"
 - Plant will not spread to a cell that has adjacent plants.

special (array of rule objects)

Represents positive effects, synergies, or system interactions.

```
{
  "type": string,
  ...optional parameters
}
```

Common Special Types

- "adjacent_maturity_boost"
 - Increases maturity speed based on number of neighbouring plants.
 - Requires:

```
{
  "type": "adjacent_maturity_boost",
  "value": int
}
```
- "adjacent_maturity_penalty"
 - Slows maturation based on number of neighbours.
 - Requires:

```
{
  "type": "adjacent_maturity_penalty",
  "value": int
}
```
- "adjacent_shade_penalty"
 - Applies negative effect under shade adjacency.
- "shade_radius"
 - Generates shade within radius.
 - Requires:

```
{
  "type": "shade_radius",
  "value": int
}
```
- "burnt_soil_radius"
 - Modifies terrain state around plants.
 - Requires:

```
{
  "type": "burnt_soil_radius",
  "value": int
}
```
- "soil_nutrient_regeneration"
 - Helps restore nutrient points to surrounding cells.
- "subsurface_growth"
 - Enables underground network propagation.
 - The subsurface plant on a cell gets promoted to the main plant if the main plant dies.

- "nutrient_transfer_network"
 - Caused by subsurface growth and enables resource sharing across connected plants. These networks also even out nutrient points across affected cells.
- "can_grow_in_shade"
 - Overrides shade restrictions.
- "animal_avoidance"
 - Plant does not feel the effects of animals present.
 - Requires parameter:


```
{
  "type": "animal_avoidance",
  "species": string
}
```
- "neighbor_spread_slowdown"
 - Reduces spread rate of nearby plants.
- "crack_spread"
 - Enables propagation across terrain cracks.
- "coexist_all_species"
 - Allows some plants to coexist on a particular cell.
- "resurrect_after_destruction"
 - Plants with this rule will reappear on a cell after some amount of ticks after they have been removed.
- "burnt_soil_maturity_boost"
 - Grows faster in burnt soil compared to other soil types
 - Requires:


```
{
  "type": "burnt_soil_maturity_boost",
  "value": int
}
```
- "boost_whiteveil_spread"
 - Boosts the spread rate of all present Whiteveil Mycelium.
 - Requires:


```
{
  "type": "boost_whiteveil_spread",
  "value": int
}
```

Plant Unlock Condition Trees

Most plants available in the game cannot be planted unless certain conditions are met.

The full set of unlock conditions for every plant species is provided in `plant_unlock_conditions.json`. Each plant unlocks when its condition tree evaluates to `true`.

A condition tree consists of:

- Logical nodes (AND, OR, NOT).
- Leaf condition nodes (coverage checks, species checks, events, counts, etc.).

Logical Nodes

AND

All child conditions must be true.

```
{
  "op": "AND",
  "children": [
    { ... },
    { ... }
  ]
}
```

OR

At least one child condition must be true.

```
{
  "op": "OR",
  "children": [
    { ... },
    { ... }
  ]
}
```

NOT

The child condition must be false.

```
{
  "op": "NOT",
  "child": { ... }
}
```

Leaf Conditions (termination nodes)

Species Present

Checks whether a species exists anywhere in the world.

```
{  
  "type": "species_present",  
  "species": "Loamcrawlers"  
}
```

Species Absent

Checks whether a species does not exist anywhere in the world.

```
{  
  "type": "species_absent",  
  "species": "Monocryx"  
}
```

Plant Coverage

Checks the percentage of cells occupied by a plant.

```
{  
  "type": "coverage",  
  "plant": "Rose Bush",  
  "operator": ">",  
  "value": 0.08  
}
```

Plant Count

Checks how many instances of a plant exist.

```
{  
  "type": "count",  
  "plant": "Purple Canopy Tree",  
  "operator": ">=",  
  "value": 4  
}
```

Event Active

Checks whether a world event has transpired at least once.

```
{  
  "type": "event",  
  "event": "Rain"  
}
```

Feature Count

Checks how many cells have a specific world feature.

```
{
  "type": "feature_count",
  "feature": "burnt_soil",
  "operator": ">=",
  "value": 20
}
```

Animal Interaction Data Structure

The data can be found in `animals.json`.

Each animal is represented as a structured rule object consisting of:

- `id`: Unique name.
- `requirements`: A boolean tree of conditions (AND/OR nodes).
- `effects`: A list of modifiers applied when requirements are satisfied.

Core Schema

Each animal follows this structure:

```
{
  "id": "string",
  "name": "string",
  "requirements": {
    "type": "AND | OR | CONDITION",
    "conditions": []
  },
  "effects": [
    {
      "type": "string",
      "target": "string",
      "value": "number | string",
      "mode": "add | multiply | set"
    }
  ]
}
```

Condition Types

All requirements are composed from these primitives:

Coverage condition (similar to coverage conditions for plant unlocking)

```
{
  "type": "coverage",
  "species": ["Lavender"],
  "threshold": 0.08,
  "operator": ">="
}
```

Count condition

```
{
  "type": "count",
  "species": "Oak Tree",
  "threshold": 8,
  "operator": ">="
}
```

The count condition may also reference a plant group via a `species_group`. These groups can be found in the `clasifications.json` file. Example:

```
{
  "type": "count",
  "species_group": ["Shallow-root Species"],
  "threshold": 25,
  "operator": ">="
}
```

Group coverage condition

```
{
  "type": "group_coverage",
  "species_group": [
    "Lavender",
    "Rose Bush",
    "Dwarf Sunflower",
    "Orange Blossom"
  ],
  "threshold": 0.12,
  "operator": ">="
}
```

Dominance condition

```
{
  "type": "dominance",
  "mode": "single_species",
  "threshold": 0.5
}
```

Final Word

We are counting on you to bring back an ideal sample of Photospherian plants. Be safe, have fun, and gooooooooood luck!

